OptiSolarAI: Sistema Inteligente de Gestión Energética para Maximizar Beneficios de la Energía Solar

Memoria actualizada y ampliada

Alumno: Diego Quiroga Bausa

Curso 2025/2026
Fecha de actualización: 19/03/2026

Índice

1. Título provisional del proyecto **4**

2. Introducción y contexto **4**

3. Objetivos **4**
3.1. Objetivo general 4
3.2. Objetivos específicos iniciales (base de memoria) 4
3.3. Objetivos específicos ampliados con lo nuevo implementado 5

4. Impacto y aspectos éticos **5**

5. Metodología y herramientas **5**
5.1. Enfoque metodológico 5
5.2. Stack técnico 6

6. Planificación inicial y estado real de avance **6**
6.1. Planificación base (memoria original) 6
6.2. Estado actualizado por fases 6

7. Funcionalidades previstas e implementación actual **7**
7.1. Funcionalidades previstas en la memoria original 7
7.2. Funcionalidades nuevas implementadas desde la entrega 7

8. Arquitectura técnica actual **7**
8.1. Estructura modular 7
8.2. Modelo de datos principal (DuckDB) 8

9. Detalle de implementaciones nuevas **8**
9.1. 1) Agente de Q-Learning 8
9.2. 2) Integración RL en flujo de simulación real 8
9.3. 3) Datos sintéticos ampliados y más realistas 9
9.4. 4) Robustez de consultas y experiencia de fallo seguro 9
9.5. 5) Evolución de interfaz y diseño 9
9.6. 6) Módulos de actualización y trazabilidad de cambios 9

10. Flujo de simulación y cálculo económico **9**
10.1. Lógica general por hora 9
10.2. Interpretación de decisiones 9

11. Machine Learning supervisado y meteorología **10**
11.1. Modelo de predicción solar 10
11.2. Entrenamiento y persistencia 10
11.3. Integración de API meteorológica 10

12. Validación, pruebas y calidad **10**
12.1. Pruebas funcionales realizadas 10
12.2. Mecanismos de calidad incorporados 10

13. Comparativa: memoria inicial vs estado actual **10**

14. Riesgos, limitaciones y trabajo pendiente **11**
14.1. Limitaciones actuales 11
14.2. Riesgos técnicos 11
14.3. Próximos pasos recomendados 11

15. Conclusión final actualizada **12**

Resumen Ejecutivo

OptiSolarAI es un sistema de apoyo a la decisión para instalaciones solares que combina analítica de datos, simulación energética y algoritmos de inteligencia artificial para mejorar la rentabilidad de la energía generada. En esta versión de memoria se mantiene la base de la propuesta inicial y se incorporan, con mayor profundidad, todas las implementaciones relevantes realizadas durante el desarrollo.

La evolución principal del proyecto respecto a la entrega inicial ha sido la incorporación de una capa de aprendizaje por refuerzo (Q-Learning) para la gestión de batería, la ampliación y mejora del dataset sintético de prueba, el robustecimiento de la capa de datos en DuckDB y la consolidación de una interfaz de simulación orientada a comparativa económica de estrategias de operación.

1. Título provisional del proyecto

OptiSolarAI: Plataforma Web Interactiva para Optimizar el Uso y Beneficio de la Energía Solar mediante Machine Learning y Aprendizaje por Refuerzo.

2. Introducción y contexto

La energía solar ha crecido de forma sostenida tanto en el sector residencial como en el empresarial. Sin embargo, disponer de generación fotovoltaica no garantiza por sí sola una explotación eficiente de la energía producida. El problema real aparece en la operación diaria: decidir cuándo consumir energía propia, cuándo almacenarla y cuándo venderla a la red.

En la práctica, muchas instalaciones operan con reglas estáticas o por intuición. Esto provoca pérdidas de oportunidad económica, especialmente cuando el precio de la electricidad varía por horas y no se adapta la estrategia de batería a ese contexto dinámico.

OptiSolarAI aborda este reto con una aplicación construida en Streamlit, estructurada en módulos de datos, lógica, predicción y visualización. El sistema permite ejecutar simulaciones operativas, cuantificar el impacto económico de las decisiones y comparar dos enfoques de control:

- Estrategia clásica por reglas heurísticas.
- Estrategia adaptativa con agente de aprendizaje por refuerzo (Q-Learning).

Esta memoria actualizada no reemplaza tu propuesta original, sino que la extiende y la aterriza sobre el estado real del repositorio y las implementaciones efectivas alcanzadas.

3. Objetivos

3.1. Objetivo general

Crear una plataforma web interactiva que permita a empresas y particulares gestionar de manera inteligente la energía solar, maximizando tanto la eficiencia energética como los beneficios económicos.

3.2. Objetivos específicos iniciales (base de memoria)

1. Desarrollar un modelo de Machine Learning que prediga la producción solar y el precio horario de la electricidad.
2. Simular automáticamente la carga, descarga y venta de energía de la batería según las predicciones del modelo.
3. Desarrollar una interfaz web en Streamlit con visualizaciones interactivas de ahorro y beneficios.
4. Integrar datos meteorológicos reales mediante API externa para mejorar predicciones.
5. Permitir configuración personalizada de batería, consumo y estrategia.

6. Ofrecer escenarios de simulación para apoyar la toma de decisiones.
7. Mostrar de forma comprensible cómo la IA influye en la gestión energética.

3.3. Objetivos específicos ampliados con lo nuevo implementado

1. Incorporar un agente Q-Learning operativo para decidir acciones de batería por estado discretizado.
2. Persistir el conocimiento del agente RL para reutilizar aprendizaje entre ejecuciones.
3. Aumentar la calidad del conjunto de datos sintéticos de test (de 7 a 30 días) para entrenar y validar mejor.
4. Endurecer la capa de datos con `try/except` y salidas seguras para evitar caídas de la app.
5. Integrar flujo de entrenamiento del agente RL en la interfaz de simulación con feedback visual.
6. Medir resultados económicos hora a hora y acumulados para comparar estrategias de control.
7. Documentar el estado real de funcionalidades implementadas frente a las inicialmente previstas.

4. Impacto y aspectos éticos

- **Sostenibilidad:** favorece el consumo eficiente de energía renovable y reduce compras en franjas de precio desfavorable.
- **Impacto económico:** cuantifica beneficios potenciales al optimizar decisiones de compra/venta/almacenamiento.
- **Privacidad:** no requiere datos personales para la simulación base; trabaja con datos sintéticos o agregados.
- **Transparencia algorítmica:** cada hora de simulación incluye decisión tomada, energía movida y resultado económico.
- **Uso responsable de IA:** la IA actúa como apoyo a decisión y no como sustitución de criterio operativo experto.
- **Licencias y ecosistema:** el desarrollo se apoya en librerías abiertas (Streamlit, DuckDB, Plotly, pandas, numpy, scikit-learn).

5. Metodología y herramientas

5.1. Enfoque metodológico

El proyecto se ha desarrollado de forma iterativa, con incrementos funcionales y validación continua sobre una base modular. Se ha seguido una lógica tipo Kanban: diseño de funcionalidad, implementación, prueba manual, ajuste y documentación.

5.2. Stack técnico

- **Lenguaje principal:** Python 3.x.
- **Interfaz:** Streamlit.
- **Visualización:** Plotly.
- **Análisis de datos:** pandas, numpy.
- **Modelo supervisado:** Random Forest Regressor (scikit-learn).
- **Aprendizaje por refuerzo:** Q-Learning tabular.
- **Base de datos:** DuckDB (persistencia local analítica).
- **Conectividad externa:** cliente OpenWeatherMap con fallback.
- **Entorno:** scripts de instalación/ejecución para Windows (`INSTALAR.bat`, `EJECUTAR.bat`).

6. Planificación inicial y estado real de avance

6.1. Planificación base (memoria original)

1. Fase 1: Definición y análisis.
2. Fase 2: Diseño del modelo de Machine Learning.
3. Fase 3: Desarrollo de interfaz web.
4. Fase 4: Integración meteorológica.
5. Fase 5: Simulación y visualización económica.
6. Fase 6: Optimización y pruebas.
7. Fase 7: Presentación final y memoria.

6.2. Estado actualizado por fases

- **Fase 1:** completada (definición de métricas y arquitectura base).
- **Fase 2:** completada (entrenamiento de Random Forest y serialización de modelo).
- **Fase 3:** completada con evolución visual y reorganización de flujo operativo.
- **Fase 4:** parcialmente completada (cliente API implementado; integración productiva mejorable).
- **Fase 5:** completada y ampliada (simulación clásica + simulación con agente RL).
- **Fase 6:** completada en robustez de consultas y manejo de errores.
- **Fase 7:** en consolidación documental y alineación final de UI con todas las capacidades ya codificadas.

7. Funcionalidades previstas e implementación actual

7.1. Funcionalidades previstas en la memoria original

- Panel principal con estado de batería, predicciones y precios.
- Gráficos de ahorro, venta y carga/descarga.
- Configuración de batería, tarifas y consumo.
- Integración meteorológica y alertas.
- Escenarios comparativos para empresas y particulares.
- Exportación de resultados.
- Visualización de cómo decide la IA.

7.2. Funcionalidades nuevas implementadas desde la entrega

1. **Agente RL en producción de simulación**: Se implementa `AgenteRL` en `rl_engine.py` y se integra en `logic.py` para seleccionar acciones según estado.
2. **Modo dual de simulación**: En `app.py` se habilita selección entre *Reglas Fijas* y *Q-Learning*, con opción de entrenamiento previo por *épocas*.
3. **Persistencia de aprendizaje**: El agente guarda la Q-table en `models/q_learning_agent.pkl` y la recupera en ejecuciones futuras.
4. **Ampliación del dataset sintético**: Se pasa de 7 días a 30 días para aumentar estabilidad de entrenamiento y validez de simulación.
5. **Mayor robustez en base de datos**: Consultas encapsuladas en `try/except` con devolución de dataframes vacíos compatibles para evitar interrupciones de la app.
6. **Métricas de resultado más completas**: Beneficio total, beneficio medio diario, carga final, energía comprada/vendida y ciclos de batería.
7. **Mejora de experiencia visual**: Tema oscuro con estilo glassmorphism, componentes KPI y gráficos adaptados al contexto de operación.
8. **Módulos de soporte consolidados**: `config.py` y `utils.py` amplían configuración centralizada, validaciones y utilidades para reporte/análisis.

8. Arquitectura técnica actual

8.1. Estructura modular

Archivo	Responsabilidad

`app.py`	UI Streamlit, selección de fechas/parámetros, entrenamiento y ejecución de simulación, visualización económica.
`database.py`	Conexión DuckDB, creación de tablas, inserciones y consultas de precios, producción, clima, consumo y estadísticas.
`ml_engine.py`	Modelo Random Forest, inferencia, pronóstico y cliente OpenWeatherMap con fallback.
`logic.py`	Simulador de batería, decisiones heurísticas y puente con agente RL.
`rl_engine.py`	Implementación de Q-Learning (estado, acción, recompensa, Bellman, persistencia).
`config.py`	Parametrización de batería, precios, financiero, visualización y validaciones.
`utils.py`	Utilidades de analítica, exportación, transformaciones y helpers de presentación.

8.2. Modelo de datos principal (DuckDB)

- `precios_luz(fecha_hora, precio_kwh)`
- `produccion_solar(fecha_hora, produccion_kwh, radiacion)`
- `clima(fecha_hora, temperatura, nubosidad, humedad)`
- `simulaciones_bateria(...)`
- `registre_consum(...)`

9. Detalle de implementaciones nuevas

9.1. 1) Agente de Q-Learning

El estado se discretiza en cuatro dimensiones: bloque horario, nivel de carga, nivel de precio y balance energético. Con esa discretización, el agente aplica una política *epsilon-greedy* y actualiza la tabla Q según Bellman.

Valor técnico aportado:

- Permite adaptación a condiciones cambiantes.
- Evita depender exclusivamente de umbrales estáticos.
- Abre puerta a comparar rendimiento de estrategia clásica frente a estrategia aprendida.

9.2. 2) Integración RL en flujo de simulación real

En interfaz se añade selector de método y entrenamiento opcional en varias *épocas*. El usuario ejecuta una simulación final tras entrenamiento y obtiene KPIs y trazas horarias.

Resultado funcional:

- IA explicable por historial de decisiones.
- Comparativa de impacto económico según estrategia.

9.3. 3) Datos sintéticos ampliados y más realistas

El sistema de carga de ejemplo genera 30 días de datos con patrones diarios y semanales para precio y producción, aportando mayor estabilidad a entrenamiento y pruebas.

Mejora conseguida:

- Menos sensibilidad a outliers de pocos días.
- Escenarios más representativos para validación.

9.4. 4) Robustez de consultas y experiencia de fallo seguro

Se implementa un enfoque de tolerancia a fallos en consultas a BD: ante error, se retornan dataframes vacíos con esquema esperado. La interfaz no colapsa y puede mostrar mensajes de estado en lugar de errores crudos.

9.5. 5) Evolución de interfaz y diseño

Se incorpora tema visual coherente con objetivo de dashboard energético avanzado: fondo degradado, tarjetas semitransparentes, mejora de legibilidad y visualización de métricas clave.

9.6. 6) Módulos de actualización y trazabilidad de cambios

El repositorio incluye scripts de actualización (`update_app.py`, `update_app_ui.py`) que evidencian la evolución incremental de la app y la introducción de nuevas capacidades sobre versiones previas.

10. Flujo de simulación y cálculo económico

10.1. Lógica general por hora

Para cada marca temporal:

1. Se calcula el balance: `energia_disponible = produccion - consumo`.
2. Se decide acción mediante heurística o RL.
3. Se ejecuta la acción respetando capacidad y eficiencias.
4. Se actualiza beneficio de la hora y beneficio acumulado.
5. Se registra el estado para trazabilidad y visualización.

10.2. Interpretación de decisiones

- `cargar`: almacenar excedente para uso o venta posterior.
- `descargar`: cubrir déficit y evitar compra cara.
- `vender`: monetizar excedente en momentos oportunos.

- `comprar`: cubrir déficit cuando batería no puede aportar.
- `mantener`: acción neutra (con reglas de seguridad sobre déficit/excedente).

11. Machine Learning supervisado y meteorología

11.1. Modelo de predicción solar

El modelo Random Forest usa variables meteorológicas (`temperatura`, `nubosidad`, `humedad`, `radiacion`) para estimar `produccion_kwh`.

11.2. Entrenamiento y persistencia

- División train/test para evaluación interna.
- Cálculo de MAE y R^2 como métricas de referencia.
- Guardado de modelo y metadatos en `models/solar_predictor.pkl`.

11.3. Integración de API meteorológica

Se implementa cliente para OpenWeatherMap con timeouts y fallback de datos de ejemplo. Esto evita bloquear la app cuando no hay API key o conectividad.

12. Validación, pruebas y calidad

12.1. Pruebas funcionales realizadas

- Carga de datos sintéticos y verificación de recuentos en BD.
- Entrenamiento de modelo ML desde interfaz.
- Ejecución de simulación con reglas fijas.
- Ejecución de simulación con agente RL y entrenamiento previo.
- Revisión de KPIs y consistencia de gráficos de beneficio/carga.

12.2. Mecanismos de calidad incorporados

- Control de errores en lectura de datos.
- Validaciones básicas en configuración y parámetros.
- Separación por módulos para reducir acoplamiento.

13. Comparativa: memoria inicial vs estado actual

Elemento de memoria inicial	Estado actual y ampliaciones
Predicción con ML	Implementada con Random Forest y persistencia de modelo.
Simulación de batería	Implementada y extendida con enfoque dual (heurístico + RL).
Interfaz interactiva	Implementada con visualizaciones económicas y panel de simulación.
Integración clima	Cliente API implementado con fallback seguro.
Escenarios de estrategia	Implementados a través de parámetros y tipo de controlador.
Monitor consumo y previsión dedicados en UI	Parcial respecto a versiones documentadas; en código existen componentes y funciones, pero la UI actual está centrada en simulación y beneficio.
Robustez de sistema	Mejorada mediante manejo de excepciones y respuestas seguras en BD.

14. Riesgos, limitaciones y trabajo pendiente

14.1. Limitaciones actuales

- El entrenamiento RL está acotado a una discretización sencilla de estados.
- La validación se realiza principalmente con datos sintéticos.
- Existe desalineación parcial entre algunas piezas documentadas históricamente y la UI activa actual.

14.2. Riesgos técnicos

- Sobreajuste del controlador si se entrena con escenarios poco variados.
- Dependencia de la calidad de datos para que la estrategia económica sea fiable.
- Diferencias entre entorno académico y despliegue real (tarifas, normativa, degradación de batería, etc.).

14.3. Próximos pasos recomendados

1. Reintegrar en UI, de forma consistente, todas las capacidades ya presentes en módulos (monitor consumo y previsión avanzada).
2. Incorporar datos reales de precio y meteorología con pipeline de ingesta estable.
3. Añadir cuadro comparativo automático entre estrategia heurística y RL por periodo.
4. Introducir exportación formal de reportes (CSV/PDF) desde interfaz principal.
5. Definir un paquete de pruebas reproducibles para defensa final del proyecto.

15. Conclusión final actualizada

OptiSolarAI mantiene la visión de la memoria original y la amplía con avances técnicos relevantes. El proyecto ha pasado de un prototipo centrado en predicción y reglas fijas a una plataforma capaz de experimentar con control inteligente adaptativo mediante aprendizaje por refuerzo.

La arquitectura modular actual permite evolucionar con rapidez y conservar trazabilidad entre datos, decisiones y resultados económicos. A nivel académico, el proyecto ya aporta una demostración completa de integración entre ciencia de datos, IA y software aplicado a energía renovable. El siguiente hito es la consolidación final de interfaz y pruebas para presentar una versión de cierre totalmente alineada entre lo implementado y lo expuesto al usuario.

Anexo A: Archivos clave del repositorio

- `app.py`
- `database.py`
- `logic.py`
- `ml_engine.py`
- `rl_engine.py`
- `config.py`
- `utils.py`
- `requirements.txt`
- `docs/UD1B_documentacion_tecnica.md`

Anexo B: Comandos de ejecución

- Crear entorno: `python -m venv venv`
- Activar entorno (Windows): `.
venv
Scripts
Activate.ps1`
- Instalar dependencias: `pip install -r requirements.txt`
- Ejecutar app: `streamlit run app.py`